

O modelo C4 de documentação para Arquitetura de Software

01 AGO 2018 • 7 MIN(S) DE LEITURA

CURTIR

por



Simon Brown

SEGUIR

traduzido por

SEGUIR

Marcelo Costa is a Solution

Pontos Principais

A criação de diagramas de software foi reduzida como resultado da mudança para o uso das metodologias ágeis. Quando diagramas são criados, eles geralmente são confusos e pouco claros.

O modelo C4 consiste em um conjunto hierárquico de diagramas de arquitetura de software para contexto, containers, componentes e código.

A hierarquia dos diagramas C4 fornece diferentes níveis de abstração, cada um dos quais é relevante para um público diferente.

Evite a ambiguidade em seus diagramas incluindo uma quantidade suficiente de texto, bem como uma chave/legenda para a notação utilizada.

Os diagramas de arquitetura de software são uma maneira fantástica de comunicar como você planeja construir um sistema de software (design inicial) ou como um sistema de software existente funciona (documentação retrospectiva, compartilhamento de conhecimento e aprendizado).

No entanto, é muito provável que a maioria dos diagramas de arquitetura de software que você tenha visto seja uma bagunça confusa de caixas e linhas. Um efeito colateral infeliz e não intencional do [Manifesto para o Desenvolvimento Ágil de Software](#) é que muitas equipes pararam ou reduziram seus esforços de diagramação e documentação, incluindo o uso de UML.

Agora, essas mesmas equipes tendem a confiar em diagramas ad hoc que desenham em um quadro branco ou montam usando ferramentas de diagramação de uso geral, como o Microsoft Visio. [Ionut Balosin](#) escreveu no ano passado "[A Arte de Elaborar Diagramas Arquiteturais](#)", que descreve uma série de problemas comuns ao fazer isso, relacionados a notações incompreensíveis e semânticas pouco claras.

(Clique na imagem para ampliar)



Diagramas ambíguos de arquitetura de software levam a mal-entendidos, o que pode desacelerar uma boa equipe. Em nossa indústria, deveríamos realmente nos esforçar para criar melhores diagramas de arquitetura de software. Depois de anos construindo software e trabalhando com equipes de todo o mundo, criei algo que chamo de "modelo C4". C4 significa contexto, containers, componentes e código - um conjunto de diagramas hierárquicos que você pode usar para descrever sua arquitetura de software em diferentes níveis de zoom, cada um útil para públicos diferentes. Pense nisso como o Google Maps para o seu código.

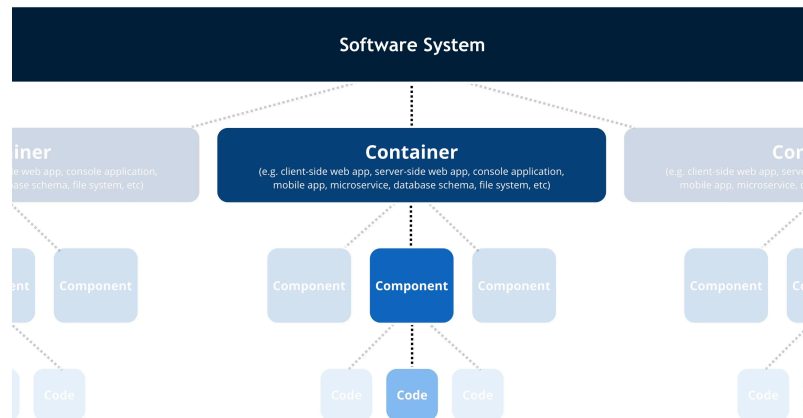
(Clique na imagem para ampliar)



Para criar esses mapas do seu código, primeiro você precisa de um conjunto comum de abstrações para criar uma linguagem onipresente para descrever a estrutura estática de um sistema de software. O modelo C4 considera as estruturas estáticas de um sistema

de software em termos de containers (aplicativos, armazenamentos de dados, microservices, etc.), componentes e código. Ele também considera as pessoas que usam os sistemas de software que construímos.

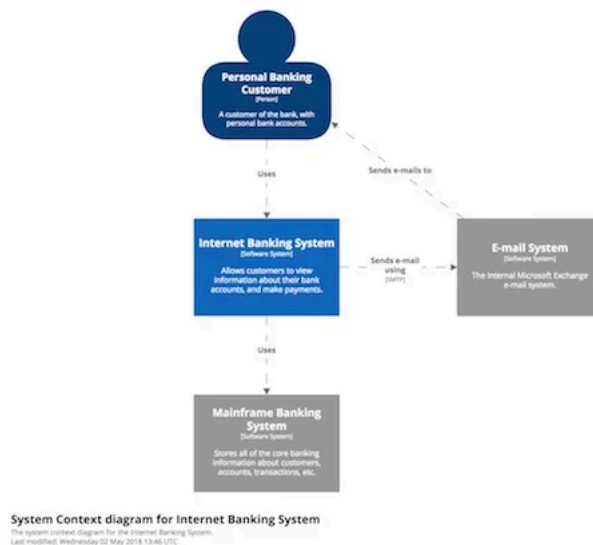
(Clique na imagem para ampliar)



O nível 1: O diagrama de contexto do sistema

O nível 1, um diagrama de contexto do sistema, mostra o sistema de software que você está construindo e como ele se encaixa no mundo em termos das pessoas que o utilizam e dos outros sistemas de software com os quais ele interage. Aqui está um exemplo de um diagrama de contexto que descreve um sistema bancário na Internet que você pode estar construindo:

(Clique na imagem para ampliar)



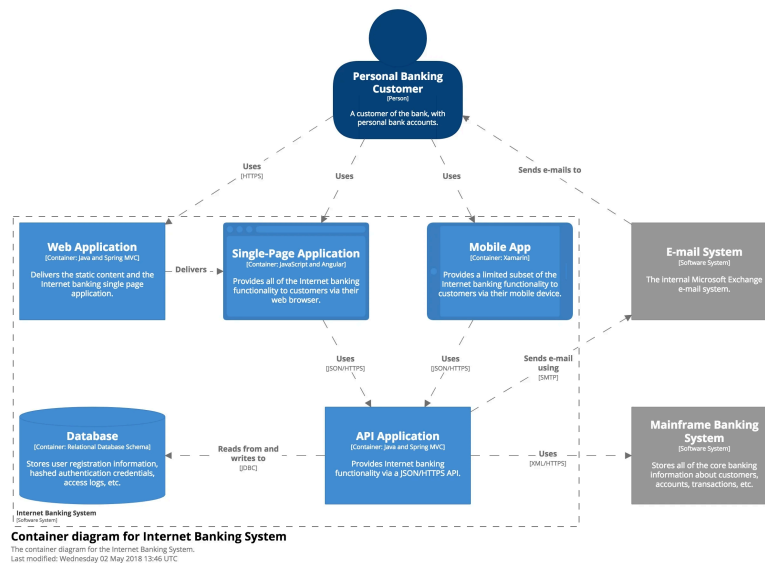
Os clientes do banco usam o sistema bancário na Internet para exibir informações sobre suas contas bancárias e efetuar pagamentos. O sistema bancário na Internet usa o sistema bancário no mainframe existente do banco para fazer isso e usa o sistema de e-mail existente do banco para enviar e-mail aos clientes. O código de cores no diagrama indica quais sistemas de software já existem (as caixas cinza) e aqueles a serem construídos (azul).

O nível 2: O diagrama de container

O nível 2, um diagrama de container, amplia o sistema de software e mostra os containers (aplicativos, armazenamentos de dados, microservices, etc.) que compõem esse sistema de software. As decisões de tecnologia também são uma parte

fundamental desse diagrama. A seguir está um diagrama de container de amostra para o sistema bancário na Internet. Isso mostra que o sistema bancário na Internet (a caixa tracejada) é composto de cinco containers: uma aplicação Web do lado do servidor, uma aplicação de página única do lado do cliente, uma aplicação móvel, uma aplicação em API do lado do servidor e um banco de dados.

(Clique na imagem para ampliar)

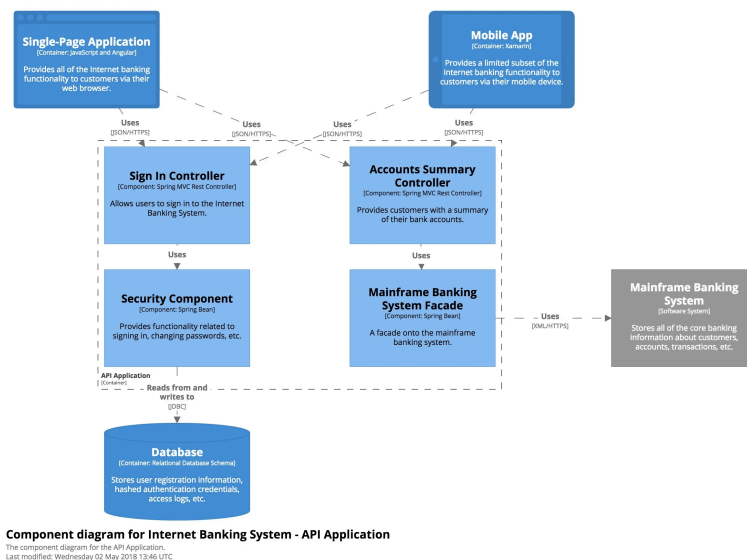


A aplicação Web é um sistema em Java/Spring MVC que simplesmente exibe conteúdo estático (HTML, CSS e JavaScript), incluindo o conteúdo que compõe a aplicação de página única. A aplicação de página única é uma aplicação Angular que é executada no navegador web do cliente, fornecendo todos os recursos do Internet banking. Como alternativa, os clientes podem usar um aplicativo móvel construído em Xamarin, cross plataforma, para acessar um subconjunto de funcionalidades do Internet banking. A aplicação de página única e o aplicativo móvel usam uma API JSON/HTTPS, que outra aplicação Java/Spring MVC executando no lado do servidor fornece. A aplicação na API obtém informações do usuário do banco de dados (um esquema de banco de dados relacional). A aplicação na API também se comunica com o sistema bancário no mainframe existente, usando uma interface proprietária XML/HTTPS, para obter informações sobre contas bancárias ou fazer transações. O aplicativo na API também usa o sistema de email existente se precisar enviar email aos clientes.

O nível 3: O diagrama de componentes

O nível 3, um diagrama de componentes, amplia um container individual para mostrar os componentes dentro dele. Esses componentes devem mapear para abstrações reais (por exemplo, um agrupamento de código) em sua base de código. Aqui está um exemplo de diagrama de componentes para o sistema bancário da Internet fictício que mostra alguns (em vez de todos) os componentes dentro da aplicação da API.

(Clique na imagem para ampliar)

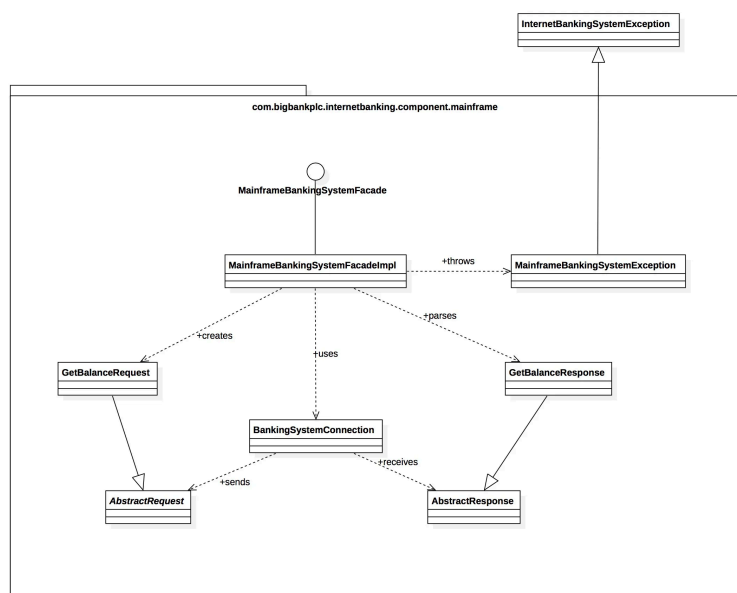


Dois controladores Spring MVC em Rest fornecem pontos de acesso para a API JSON/HTTPS, com cada controlador subsequentemente usando outros componentes para acessar dados do banco de dados e do sistema bancário no mainframe.

O nível 4: O código

Finalmente, se você realmente quiser ou precisar, você pode ampliar um componente individual para mostrar como esse componente é implementado. Este é um diagrama de classes UML de amostra (e parcial) para o sistema bancário na Internet fictício que, mostrando os elementos de código (interfaces e classes) que compõem o componente MainframeBankingSystemFacade.

(Clique na imagem para ampliar)



Esse diagrama mostra que o componente é composto por várias classes, com os detalhes da implementação refletindo diretamente o código. Eu não recomendaria necessariamente a criação de diagramas neste nível de detalhe, especialmente quando você pode obtê-los sob demanda da maioria dos IDEs.

A notação

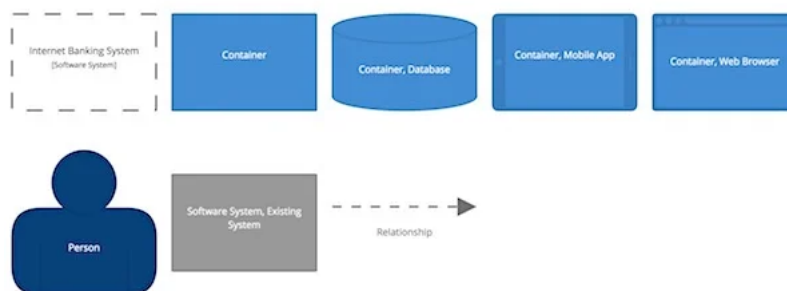
O modelo C4 não prescreve nenhuma notação específica, e o que você vê nesses diagramas apresentados é uma notação simples que funciona bem em quadros

brancos, papéis, notas adesivas, cartões de índice e uma variedade de ferramentas de diagramação. Você também pode usar a UML como sua notação, com o uso apropriado de pacotes, componentes e estereótipos.

Independentemente da notação que você usa, recomendo que cada elemento inclua um nome, o tipo de elemento (ou seja, "Pessoa", "Sistema de Software", "Container" ou "Componente"), uma opção de tecnologia (se apropriado) e algum texto descritivo. Pode parecer incomum incluir tanto texto em um diagrama, mas esse texto adicional elimina grande parte da ambiguidade tipicamente vista em diagramas de arquitetura de software.

Certifique-se de que você tenha uma chave/legenda para descrever qualquer notação que esteja usando, mesmo que seja óbvio para você. Isso deve abranger cores, formas, acrônimos, estilos de linha, bordas, dimensionamento, etc. Sua notação deve idealmente permanecer consistente em cada nível de detalhe. A seguir, apresento a chave/legenda do diagrama para o diagrama de container mostrado anteriormente.

(Clique na imagem para ampliar)



E finalmente, não se esqueça do título do diagrama, que deve aparecer em cada diagrama para descrever inequivocamente o tipo e o escopo de cada diagrama (por exemplo, "Diagrama de contexto do sistema para o sistema bancário na Internet").

Mais Informações

O modelo C4 é uma maneira simples de comunicar a arquitetura de um software em diferentes níveis de abstração, para que você possa contar histórias diferentes para diferentes públicos. É também uma maneira de introduzir (muitas vezes, reintroduzir) algum rigor e modelagem leve para as equipes de desenvolvimento de software. Acesse c4model.com para obter mais informações sobre o Modelo C4, bem como diagramas suplementares (tempo de execução e implantação), exemplos, uma lista de verificação de notação, FAQs, vídeos de palestras de conferência e opções de ferramentas.

Sobre o Autor



Simon Brown é consultor independente especializado em arquitetura de software e autor do livro "Arquitetura de Software para Desenvolvedores" (um guia amigável para desenvolvedores de arquitetura de software, liderança técnica e equilíbrio com agilidade). Ele também é o criador do modelo de arquitetura de software C4, que é uma abordagem simples para criar mapas do seu código. Simon é um orador regular nas conferências internacionais de desenvolvimento de software e viaja pelo mundo para ajudar as organizações a visualizar e documentar sua arquitetura de software.